INSTITUTO CERVANTES

Carrera Analista Programador · Cátedra Práctica Profesionalizante

DOCUMENTACIÓN DEL TRABAJO PRÁCTICO

Everyone Needs To Smile

Plataforma de peticiones vecinales con procesamiento de datos y medición de impacto

CÁTEDRA Práctica Profesionalizante	DOCENTE Ing. Loza	DIRECTOR DEL PROYECTO Andrada, Juan Pablo
CONTENIDO Documentación y entregables	VERSIÓN 4.0	SITIO PUBLICADO entscordoba.netlify.app

AUTORES

Andrada, Juan Pablo
Legajo 27318

Patrón, Máximo
Legajo 27311

Jiménez, Ignacio
Legajo 28177

Historial de revisión

Versión	Fecha y hora	Autor	Detalle de la modificación
1.0	Sprint 0 (*)	Equipo	Propuesta original: plataforma solidaria de donaciones (Project Charter, alcance, EDT, plan de comunicación, Gantt, product backlog y arquitectura).
2.0	12/10/2026 16:42	Andrada, Juan Pablo	Reorientación del proyecto: el módulo de pagos y donaciones se reemplaza por un sistema de peticiones con clasificación automática, firmas, score de urgencia, detección de similares y reportes. Documentación del sistema implementado.
3.0	12/10/2026 21:32	Andrada, Juan Pablo	Revisión integral para la entrega: glosario al comienzo, responsables y horas, rol moderador y permisos por panel, requisitos e historias de usuario con trazabilidad, sprints en el Gantt y eliminación de todos los campos pendientes.
4.0	18/10/2026 16:10	Andrada, Juan Pablo	Barrio del vecino: cada cuenta elige dónde vive y ve primero lo de su barrio (requisitos RF21 y RF22, historia HU14 y nuevos términos del glosario). Se registran los datos de demostración y la limpieza del código.

(*) La versión 1.0 se elaboró durante el Sprint 0, antes de que el equipo usara control de versiones, por lo que su fecha y hora exactas no quedaron registradas. Desde la versión 2.0, cada cambio se registra con fecha, hora y autor.

Tabla de contenido

Glosario

1. Introducción

2. Objetivos

3. Contenido del trabajo práctico

- 3.1 Contexto, estudio de necesidades y reorientación
- 3.2 Project Charter
- 3.3 Declaración de alcance (Scope Statement)
- 3.4 Estructura de desglose del trabajo (EDT / WBS)
- 3.5 Lista de actividades
- 3.6 Diccionario de actividades
- 3.7 Plan de comunicación
- 3.8 Diagrama de Gantt
- 3.9 Requerimientos y product backlog
- 3.10 Arquitectura del sistema
- 3.11 Modelo de datos
- 3.12 Procesamiento automático de datos
- 3.13 Autenticación, roles y seguridad
- 3.14 Reportes y evaluación de impacto
- 3.15 Pruebas y calidad
- 3.16 Ambiente de implementación y despliegue

4. Bibliografía

5. Fuentes de información

6. Anexo

Glosario

Siglas y términos que se usan en este documento, ordenados alfabéticamente. Cada uno se explica en lenguaje simple para que el documento pueda leerse sin conocimientos técnicos previos.

Término	Significado
Accesibilidad	Posibilidad de que cualquier persona, incluso con discapacidad, use el sistema (teclado, contraste, lectores de pantalla).
Actividad	Trabajo concreto que produce un entregable. Se identifica con un código A1, A2…
Administrador	Rol que modera, gestiona usuarios y áreas, y accede al panel analítico.
Anonimato	Regla por la que un vecino nunca ve quién creó o firmó una petición.
API	Interfaz de programación: conjunto de operaciones que un sistema ofrece a otros programas.
Área	Dependencia del municipio responsable de un grupo de categorías (por ejemplo, Salud u Obras públicas).
Bandeja	Pantalla donde cada funcionario ve las peticiones de su área ordenadas por prioridad para responderlas.
Barrio (zona)	Cada uno de los 111 barrios de Córdoba y Villa Allende cargados en el sistema con su ubicación.
Barrio del vecino (mi barrio)	Barrio donde vive cada vecino, elegido por él mismo. Ordena lo que ve primero en la plataforma y solo se muestra a otros si la persona lo permite.
Bootstrap	Biblioteca de estilos para construir interfaces adaptables a celular.
CDN	Red de distribución de contenido: servidores repartidos que entregan el sitio desde el lugar más cercano al usuario.
Criterio de aceptación	Condición verificable que debe cumplirse para dar por terminado un requisito, historia o actividad.
CSV / Excel	Formatos de planilla de cálculo usados para exportar reportes.
Datos de demostración	Cuentas, peticiones y fotos ficticias cargadas para mostrar la plataforma con actividad; se identifican con el dominio demo.ents.invalid y se borran con un solo script.
Dependencia fin-comienzo	Tipo de relación en la que una actividad empieza cuando terminan todas sus predecesoras.
DER	Diagrama entidad-relación: dibujo de las tablas de la base y cómo se relacionan.
Día hábil	Día de trabajo del equipo (lunes a viernes, sin feriados). Las duraciones del cronograma se expresan en días hábiles.
Diagrama de Gantt	Gráfico que muestra cada actividad como una barra a lo largo del tiempo.

Término	Significado
Disparador (trigger)	Función que la base ejecuta sola cuando ocurre un cambio (por ejemplo, recalcular el score al recibir una firma).
Dueño (CEO)	Cuenta única del responsable del proyecto: además de administrar, cambia roles y define los permisos de cada rol.
EDT / WBS	Estructura de desglose del trabajo (en inglés, Work Breakdown Structure): descomposición jerárquica del proyecto en entregables y paquetes de trabajo.
Elevación	Aviso automático a los funcionarios del área cuando una petición alcanza 50 firmas.
Entregable	Resultado verificable que produce el proyecto (un documento, un módulo del sistema, una presentación).
ENTS	Everyone Needs To Smile ("Todos necesitan sonreír"): nombre del proyecto y de la plataforma.
Firma	Apoyo de un vecino a una petición publicada. Cada persona puede firmar una sola vez y retirar su firma.
FK	Foreign key (clave foránea): columna que apunta a la clave primaria de otra tabla y crea la relación entre ambas.
Funcionario	Rol de un empleado del municipio: responde las peticiones de su área. Lo aprueba un administrador.
Geolocalización	Obtención de la ubicación del dispositivo (con permiso de la persona) para completar el barrio automáticamente.
GitHub	Plataforma donde se guarda el código con su historial de cambios (repositorio).
Hito	Momento clave del proyecto sin duración (por ejemplo, "primera versión funcional"). Se dibuja con un rombo.
Holgura	Días que una actividad puede atrasarse sin atrasar la fecha de fin del proyecto.
HTTPS	Protocolo de comunicación cifrado entre el navegador y el servidor.
HU	Historia de usuario: necesidad escrita desde el punto de vista del usuario con la forma "Como [rol], quiero [acción] para [beneficio]". Se numeran HU01, HU02…
ID	Identificador: código único de cada elemento de una lista (por ejemplo, RF01, HU03 o A7).
Interesado	Persona u organización afectada por el proyecto o que influye en él (en inglés, stakeholder).
JavaScript	Lenguaje de programación de las páginas web; en él está escrita la aplicación.
Leaflet	Biblioteca que dibuja mapas interactivos en la web.
Ley 25.326	Ley argentina de Protección de los Datos Personales.
Lint	Revisión automática del código que detecta errores y malas prácticas (en ENTS, con oxlint).
MDN Web Docs	Documentación de referencia de tecnologías web publicada por Mozilla.

Término	Significado
Moderación	Revisión de las peticiones y comentarios para que no se publique contenido ofensivo ni datos personales.
Moderador	Rol que revisa peticiones, comentarios y publicaciones; no administra usuarios.
Modo oscuro	Esquema de colores con fondo oscuro que reduce el brillo de la pantalla.
MVC	Modelo-vista-controlador: forma de separar el sistema en datos (modelo), pantallas (vista) y lógica que los conecta (controlador).
Netlify	Servicio que publica el sitio web con HTTPS cada vez que se actualiza el repositorio.
Node.js	Entorno que ejecuta JavaScript fuera del navegador; se usa para desarrollar y compilar la aplicación.
Nominatim	Servicio de búsqueda de OpenStreetMap que convierte nombres de lugares en coordenadas.
OAuth	Protocolo que permite ingresar con una cuenta de otro servicio (en ENTS, Google) sin compartir la contraseña.
ONG	Organización no gubernamental: organización social sin fines de lucro.
OpenStreetMap	Mapa colaborativo y gratuito del que se obtienen el mapa de fondo y la ubicación de los barrios.
Panel analítico	Pantalla de administración con indicadores de uso, moderación y participación, exportable a Excel y PDF.
Paquete de trabajo	Último nivel de la EDT: un entregable concreto que se puede asignar, estimar y verificar.
PDF	Formato de documento portable que se ve igual en cualquier dispositivo.
Permisos por panel	Configuración que define, para cada rol, qué paneles puede ver y en cuáles puede modificar. Solo la cambia el dueño.
Petición	Pedido que un vecino publica sobre un problema de su barrio (por ejemplo, un bache o falta de alumbrado).
pg_trgm	Extensión de PostgreSQL que compara textos por trigramas (grupos de 3 letras) para encontrar peticiones parecidas.
PK	Primary key (clave primaria): columna que identifica cada fila de una tabla en forma única.
PMBOK	Guía de los fundamentos para la dirección de proyectos del Project Management Institute; base de la metodología de los entregables.
PostgreSQL	Motor de base de datos relacional de código abierto que usa ENTS.
Predecesora / sucesora	Una actividad predecesora debe terminar antes de que empiece su sucesora.
Prioridad por firmas	Nivel de una petición según su cantidad de firmas: nueva (0–4), baja (5–9), media (10–19), alta (20–49) y máxima (50 o más).

Término	Significado
Product backlog	Lista priorizada de todo lo que hay que construir en el producto, expresada en historias de usuario.
Project Charter	Acta de constitución: documento que autoriza formalmente el proyecto y fija su propósito, objetivos y recursos.
Prueba automática	Programa que verifica que una parte del código da el resultado esperado, sin intervención manual.
Pull Request (PR)	Pedido para integrar los cambios de una rama a la principal, que otro integrante revisa antes de aceptar.
PWA	Progressive Web App (aplicación web progresiva): sitio web que se puede instalar en el celular y abrir como una aplicación.
Rama (branch)	Copia paralela del código para trabajar un cambio sin afectar la versión principal (main).
React	Biblioteca de JavaScript con la que se construye la interfaz de ENTS.
Repositorio	Lugar donde se guarda el código fuente con todo su historial de versiones.
Restricción	Límite que el proyecto no puede superar (plazo, presupuesto, equipo, normas).
Retrospectiva	Reunión al final de cada sprint para analizar qué mejorar en la forma de trabajar.
RF	Requisito funcional: algo que el sistema debe hacer (por ejemplo, "crear peticiones con fotos"). Se numeran RF01, RF02…
Riesgo	Evento incierto que, si ocurre, afecta al proyecto. Se evalúa por su probabilidad y su impacto.
RLS	Row Level Security (seguridad por fila): reglas de la base de datos que deciden qué filas puede ver o modificar cada usuario.
RNF	Requisito no funcional: una cualidad que el sistema debe cumplir (seguridad, rendimiento, accesibilidad…). Se numeran RNF01, RNF02…
Rol (rango)	Tipo de cuenta que define qué puede hacer cada usuario: vecino, funcionario, moderador, administrador o dueño.
RPC	Remote Procedure Call: función de la base de datos que la aplicación invoca para operaciones sensibles, verificando el rol adentro de la base.
Ruta crítica	Secuencia de actividades sin holgura: si una se atrasa, se atrasa todo el proyecto.
Score de urgencia	Puntaje de 0 a 100 que calcula el sistema según la gravedad de la categoría, palabras de riesgo, firmas, velocidad de apoyo y fotos.
Scrum	Marco de trabajo ágil que organiza el desarrollo en ciclos cortos (sprints) con reuniones de planificación, revisión y retrospectiva.
Seguimiento	Etapas por las que pasa una petición publicada: recibida, en revisión, derivada, en obra y resuelta.
Service worker	Programa que el navegador ejecuta en segundo plano y permite que la app abra aun con mala conexión.

Término	Significado
SMS	Mensaje de texto por celular.
Sprint	Ciclo de trabajo de duración fija (en este proyecto, 2 semanas) al final del cual se muestra un avance funcionando.
SQL	Lenguaje estándar para definir y consultar bases de datos relacionales.
Supabase	Servicio en la nube que provee la base de datos PostgreSQL, la autenticación y el almacenamiento de archivos.
Supuesto	Condición que se da por cierta para planificar, aunque no esté comprobada.
Vecino	Rol base: crea, firma y comenta peticiones. Para los demás vecinos es anónimo.
Versión del documento	Número que cambia con cada modificación registrada en el historial de revisión (1.0, 2.0, 3.0, 4.0).
Vite	Herramienta que compila y sirve la aplicación durante el desarrollo.
Vitest	Herramienta con la que se escriben y ejecutan las pruebas automáticas.
WCAG	Web Content Accessibility Guidelines: pautas internacionales de accesibilidad web del W3C.

1 Introducción

Everyone Needs To Smile (ENTS) es una plataforma web en la que los vecinos de la ciudad de Córdoba y de Villa Allende publican problemas de su barrio —salud, seguridad, infraestructura, servicios, vivienda, ambiente, entre otros—, los apoyan con su firma y siguen cómo los resuelven los organismos responsables.

El proyecto nació como una plataforma solidaria de donaciones. Durante el análisis, el equipo concluyó que el módulo de pagos agregaba complejidad desproporcionada (integración con pasarelas, cumplimiento legal y trazabilidad de fondos) frente al valor que aportaba. Por eso se reorientó hacia un **sistema de peticiones ciudadanas con medición de impacto**: no un formulario de reclamos, sino un sistema que procesa los datos —clasificación automática por categoría y zona, score de urgencia, detección de peticiones similares y reportes exportables— para que los organismos sepan qué atender primero y puedan registrar la respuesta, cerrando el ciclo.

Este documento presenta los entregables de gestión del proyecto actualizados al nuevo alcance (Project Charter, declaración de alcance, EDT, lista y diccionario de actividades, plan de comunicación y diagrama de Gantt) y la documentación técnica del sistema efectivamente implementado y publicado: requerimientos, arquitectura, modelo de datos, algoritmos de procesamiento, seguridad, pruebas y ambiente de implementación.

Consideramos que el trabajo es relevante porque convierte reclamos dispersos en información ordenada por barrio, urgencia y apoyo vecinal, que tanto la comunidad como las áreas del municipio pueden consultar, priorizar y responder.

2 Objetivos

2.1 Objetivo general

Desarrollar y poner en funcionamiento una plataforma web de peticiones vecinales para Córdoba y Villa Allende que permita a los vecinos visibilizar problemáticas de su barrio, apoyarlas con firmas y seguir su resolución, y que procese esa información para que los organismos públicos y las organizaciones sociales prioricen y registren su respuesta.

2.2 Objetivos específicos

1. Permitir crear peticiones con texto, fotos y ubicación, clasificándolas automáticamente por categoría y barrio.
2. Registrar firmas de apoyo y ordenar las peticiones por prioridad según la cantidad de firmas.
3. Calcular automáticamente un score de urgencia de 0 a 100 a partir de la gravedad, el riesgo, el apoyo y su velocidad.
4. Detectar peticiones similares para concentrar el apoyo en un único pedido.
5. Ofrecer a los funcionarios una bandeja por área para responder y registrar el avance hasta la resolución.
6. Generar estadísticas públicas anónimas y reportes exportables (Excel y PDF) para organismos.
7. Garantizar el anonimato entre vecinos y la protección de datos personales mediante reglas aplicadas en la base de datos.
8. Asegurar la calidad con pruebas automáticas, accesibilidad y buen rendimiento en celulares.

3 Contenido del trabajo práctico

3.1 Contexto, estudio de necesidades y reorientación

Análisis socioeconómico de Córdoba y encuesta

El equipo realizó previamente un análisis socioeconómico de la ciudad de Córdoba y una encuesta de necesidades a vecinos. Ambos se mantienen como base del proyecto, ya que justifican las categorías de problemáticas, el foco en el nivel de barrio y la necesidad de un canal ordenado de reclamos.

El cuestionario, las respuestas y el análisis completos forman parte de la versión 1.0 del proyecto (documento "Everyone Needs To Smile", Sprint 0). De ellos se tomaron tres conclusiones que guían esta versión: los problemas se viven y se reclaman a nivel de barrio; los vecinos no saben qué pasa con sus reclamos; y los mismos temas —salud, seguridad, infraestructura, servicios y ambiente— se repiten en distintas zonas. Esas conclusiones dan origen a las 16 categorías de petición, al enfoque por barrio y al seguimiento visible de cada respuesta.

Necesidades detectadas

- **Vecinos:** no tienen un canal único para reclamar ni saben si alguien atiende su pedido; reclaman en forma individual y sin seguimiento.
- **Barrios:** los mismos problemas se reportan muchas veces y de manera dispersa, sin un registro común que muestre su magnitud.
- **Organismos y ONG:** carecen de información ordenada por zona, urgencia y apoyo para decidir qué atender primero y rendir cuentas de lo hecho.

Reorientación del proyecto

Se mantiene	Se reemplaza
Análisis socioeconómico de Córdoba	Módulo de pagos y donaciones → **sistema de peticiones**
Arquitectura en capas (MVC)	Lógica de recaudación → **firmas y prioridad**
Base de datos relacional SQL	Seguimiento de fondos → **seguimiento de la respuesta oficial**
Autenticación y seguridad	Verificación de causas → **moderación y detección de similares**
Enfoque de evaluación de impacto	Informes de recaudación → **reportes para organismos**
Encuesta y estudio de necesidades	— (se agregan clasificación automática y score de urgencia)

3.2 Project Charter

Nombre del proyecto	ENTS — Everyone Needs To Smile

Descripción	Plataforma web de peticiones vecinales para Córdoba y Villa Allende, con clasificación automática, firmas, score de urgencia, detección de similares, seguimiento de la respuesta oficial y reportes para organismos.
Propósito / justificación	Dar a los vecinos un canal único y transparente para visibilizar problemas de su barrio y a los organismos información ordenada para priorizarlos, con la respuesta registrada a la vista de todos.
Objetivos medibles	• Plataforma publicada y accesible desde la web y como app instalable. • 100% de las peticiones nuevas con categoría, barrio y score asignados automáticamente. • Cobertura de los 111 barrios cargados (Córdoba y Villa Allende). • Pruebas automáticas aprobadas antes de cada publicación.
Alcance (resumen)	Registro e ingreso; peticiones con fotos y ubicación; firmas; procesamiento automático; mapa por barrios; comunidad y eventos; seguimiento y notificaciones; moderación; roles; estadísticas y reportes. Ver 3.3.
Entregables principales	Aplicación web publicada; base de datos con esquema documentado; repositorio de código; documentación del proyecto; presentación.
Interesados	Vecinos de Córdoba y Villa Allende; áreas del municipio y funcionarios; ONG y organizaciones sociales; docente de la cátedra; equipo de desarrollo.
Equipo	Andrada, Juan Pablo: director del proyecto · análisis, diseño y desarrollo Patrón, Máximo: configuración inicial · desarrollo y calidad Jiménez, Ignacio: relevamiento y comunicación con interesados
Recursos	Humanos: 3 integrantes, ≈ 300 horas estimadas en actividades (ver Entregable 1, sección 11.2). Tecnológicos: GitHub (repositorio), Supabase (base de datos, autenticación y archivos), Netlify (publicación), OpenStreetMap (mapas), planes gratuitos. Costo monetario directo: sin costo con los planes gratuitos actuales.
Hitos	Reorientación aprobada · Diseño de datos · Primera versión funcional · Sistema de roles y anonimato · Procesamiento y reportes · Calidad (pruebas, rendimiento) · Entrega final.
Riesgos principales	Baja participación de vecinos al inicio · Contenido inapropiado o con datos personales · Límites de los planes gratuitos · Dependencia de servicios externos · Disponibilidad del equipo.
Supuestos	Los usuarios cuentan con acceso a internet y un navegador moderno; los organismos designan funcionarios para responder.
Restricciones	Plazo del ciclo lectivo; equipo de 3 personas; sin presupuesto; sin integración con sistemas internos del municipio.
Criterios de éxito	El ciclo completo (pedir → firmar → priorizar → responder → resolver) funciona de punta a punta en el sitio publicado, con las reglas de privacidad verificadas.
Aprobación	Docente: _____ Fecha: ___/___/_____

3.3 Declaración de alcance (Scope Statement)

Descripción del producto

Aplicación web adaptable a celular e instalable (PWA), con un panel para vecinos, una bandeja para funcionarios y paneles de moderación y análisis para administradores, respaldada por una base de datos relacional con reglas de seguridad por fila.

Incluido en el alcance

- Registro, ingreso con correo o Google y recuperación de contraseña.
- Peticiones con hasta 3 fotos y ubicación en mapa.
- Categoría y barrio automáticos; score de urgencia automático.
- Firmas, prioridad por firmas y meta de 50 firmas (elevación).
- Detección de peticiones similares.
- Moderación previa a la publicación y revisión automática de textos.
- Seguimiento en 5 etapas con respuesta oficial y foto.
- Comentarios anónimos con denuncias.
- Notificaciones dentro de la app y barrios seguidos.
- Mapa por barrios, comunidad vecinal y agenda de eventos.
- Barrio del vecino: cada cuenta elige dónde vive y ve primero lo de su barrio.
- Estadísticas públicas, panel analítico y exportación a Excel y PDF.
- Roles: vecino, funcionario por área, moderador, administrador y dueño, con permisos por panel definidos por el dueño.

Fuera del alcance

- Pagos, donaciones o recaudación dentro de la plataforma (si existen, se realizan por fuera).
- Integración con sistemas internos del municipio.
- Envío de notificaciones por correo o SMS.
- Aplicación nativa de tienda (se cubre con la PWA).
- Polígonos oficiales de barrios (se usan centros de OpenStreetMap).

Criterios de aceptación

- Cada funcionalidad incluida operativa en el sitio publicado.
- Un vecino no puede ver identidades ajenas ni datos de otros.
- Pruebas automáticas aprobadas y compilación sin errores.

Supuestos y restricciones

Se trabaja con los planes gratuitos de los servicios elegidos, un equipo de tres integrantes y el calendario de la cátedra. La información geográfica de los barrios proviene de OpenStreetMap.

3.4 Estructura de desglose del trabajo (EDT / WBS)

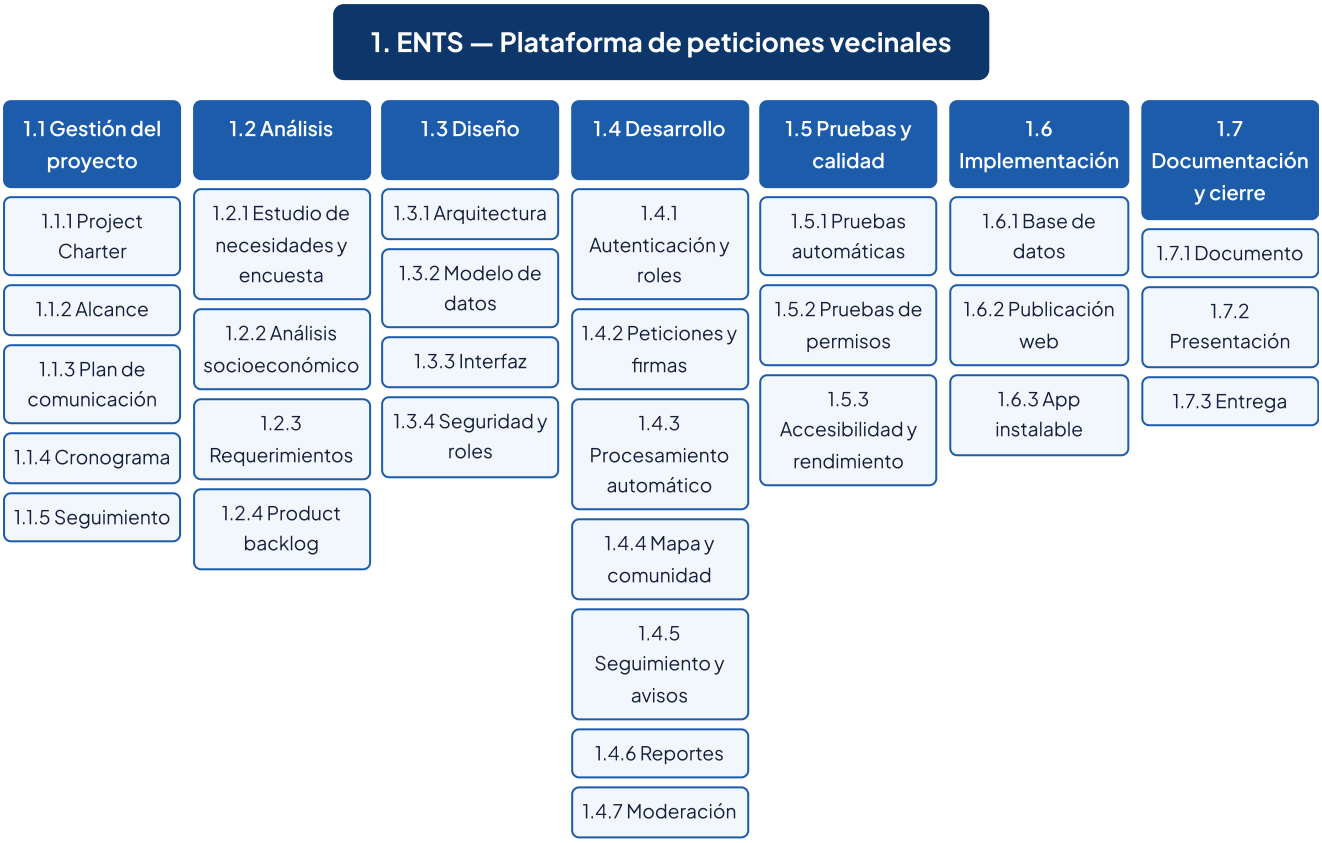


Figura 1. EDT del proyecto orientada a entregables (tres niveles).

3.5 Lista de actividades

Duraciones en días hábiles de trabajo del equipo. Las predecesoras indican qué actividades deben terminar antes de empezar la siguiente (dependencia fin-comienzo). En negrita, las actividades de la ruta crítica. Entre paréntesis, los integrantes que participan junto al responsable.

ID	Actividad	EDT	Duración	Predecesoras	Responsable
A1	Analizar la propuesta de reorientación y redefinir el proyecto	1.1.1	3	—	Andrada (Patrón, Jiménez)
A2	Redactar Project Charter y declaración de alcance	1.1.1, 1.1.2	3	A1	Andrada (Jiménez)
A3	Definir plan de comunicación y cronograma	1.1.3, 1.1.4	2	A2	Jiménez (Andrada)
A4	Revisar encuesta y análisis socioeconómico	1.2.1, 1.2.2	3	A1	Jiménez
A5	Relevar requerimientos y armar product backlog	1.2.3, 1.2.4	4	A2, A4	Andrada (Jiménez)
A6	Diseñar arquitectura y elegir tecnologías	1.3.1	3	A5, A3	Andrada (Patrón)

ID	Actividad	EDT	Duración	Predecesoras	Responsable
A7	Diseñar modelo de datos y reglas de seguridad	1.3.2, 1.3.4	4	A6	Andrada
A8	Diseñar interfaz y navegación	1.3.3	4	A5	Andrada (Jiménez)
A9	Configurar repositorio, base de datos y publicación	1.6.1, 1.6.2	2	A6	Patrón
A10	Desarrollar autenticación, roles y anonimato	1.4.1	5	A7, A9	Patrón (Andrada)
A11	Desarrollar peticiones, fotos, firmas y moderación	1.4.2, 1.4.7	6	A10, A8	Patrón (Andrada)
A12	Desarrollar clasificación, barrio, score y similares	1.4.3	5	A11	Andrada
A13	Desarrollar mapa por barrios, comunidad y eventos	1.4.4	5	A11	Andrada
A14	Desarrollar seguimiento, notificaciones y comentarios	1.4.5	5	A11	Andrada
A15	Desarrollar estadísticas, panel analítico y exportes	1.4.6	4	A12, A13	Andrada
A16	Pruebas automáticas y pruebas de permisos por rol	1.5.1, 1.5.2	4	A12, A14, A15	Patrón (Andrada)
A17	Accesibilidad, modo oscuro, app instalable y rendimiento	1.5.3, 1.6.3	4	A15	Andrada (Patrón)
A18	Redactar documentación	1.7.1	5	A16, A17	Andrada (Jiménez)
A19	Preparar presentación	1.7.2	2	A18	Jiménez (Andrada, Patrón)
A20	Entrega y defensa	1.7.3	1	A19	Andrada (Patrón, Jiménez)

3.6 Diccionario de actividades

EDT	Paquete	Descripción	Entregable	Criterio de aceptación
1.1	Gestión del proyecto	Planificación, coordinación y control del proyecto.	Charter, alcance, plan de comunicación, Gantt	Aprobados por el equipo y la cátedra
1.2.1	Estudio de necesidades	Encuesta a vecinos y análisis de problemáticas por barrio.	Informe de resultados	Necesidades priorizadas y vinculadas a categorías

EDT	Paquete	Descripción	Entregable	Criterio de aceptación
1.2.3	Requerimientos	Relevamiento de requerimientos funcionales y no funcionales.	Lista RF/RNF	Cada RF trazable a una historia de usuario
1.2.4	Product backlog	Historias de usuario con criterios de aceptación y prioridad.	Backlog priorizado	Historias estimables y verificables
1.3.1	Arquitectura	Definición de capas, tecnologías y despliegue.	Diagrama de arquitectura y despliegue	Cubre todos los RF y RNF
1.3.2	Modelo de datos	Tablas, relaciones, restricciones y funciones de la base.	DER y esquema SQL	Esquema reproducible desde `schema.sql`
1.3.4	Seguridad y roles	Reglas de acceso por fila y por columna, roles y anonimato.	Políticas RLS y funciones	Pruebas por rol sin accesos indebidos
1.4.1	Autenticación y roles	Registro, ingreso con Google, recuperación, solicitud de funcionario.	Módulo de cuentas	Flujos completos funcionando
1.4.2	Peticiones y firmas	Alta con fotos y ubicación, firma y retiro de firma, prioridad.	Módulo de peticiones	Un usuario firma una sola vez; contador correcto
1.4.3	Procesamiento automático	Clasificación, barrio, score de urgencia y similares.	Algoritmos y funciones	Casos de prueba resueltos correctamente
1.4.4	Mapa y comunidad	Mapa por barrios, muro vecinal y eventos.	Pantallas de mapa, comunidad y agenda	Filtros por barrio operativos
1.4.5	Seguimiento y avisos	Etapas, respuesta oficial, notificaciones, comentarios.	Línea de tiempo y campana	Cada cambio genera aviso a interesados
1.4.6	Reportes	Estadísticas públicas, panel analítico y exportes.	Paneles y archivos CSV/PDF	Datos coinciden con la base
1.4.7	Moderación	Aprobación, edición y revisión automática de contenido.	Panel de moderación	Nada se publica sin aprobación
1.5	Pruebas y calidad	Pruebas automáticas, de permisos, accesibilidad y rendimiento.	Suite de pruebas	100% de pruebas aprobadas
1.6	Implementación	Base de datos en la nube, publicación continua y PWA.	Sitio publicado	Cada versión publicada desde el repositorio

EDT	Paquete	Descripción	Entregable	Criterio de aceptación
1.7	Documentación y cierre	Documento, presentación y entrega.	PDF y presentación	Cumple la estructura de la cátedra

3.7 Plan de comunicación

El plan asegura que el equipo, la cátedra y los usuarios piloto reciban la información correcta, en el momento adecuado y por el canal apropiado.

Qué se comunica	Emisor	Receptor	Canal	Frecuencia	Propósito
Avance y tareas del sprint	Equipo	Equipo	Reunión de seguimiento + WhatsApp	Lunes, miércoles y viernes	Coordinar y destrabar tareas
Cambios de código	Integrante	Equipo	GitHub (ramas y Pull Requests)	Por cambio	Revisión antes de publicar
Planificación de sprint	Equipo	Equipo	Google Meet + Trello	Inicio de sprint (cada 2 semanas)	Priorizar el backlog
Revisión de sprint	Equipo	Docente	Clase / demostración	Fin de sprint	Validar avances
Cambios de alcance	Equipo	Docente	Reunión + documento	Cuando ocurren	Aprobar reorientaciones
Novedades de la plataforma	Plataforma	Vecinos	Notificaciones en la app	Automática	Informar estado de peticiones
Respuestas oficiales	Funcionarios	Vecinos	Seguimiento de la petición	Por actualización	Cerrar el ciclo de la petición
Reportes de problemáticas	Administración	Organismos y ONG	Exportes Excel / PDF por correo	Mensual	Priorizar y rendir cuentas

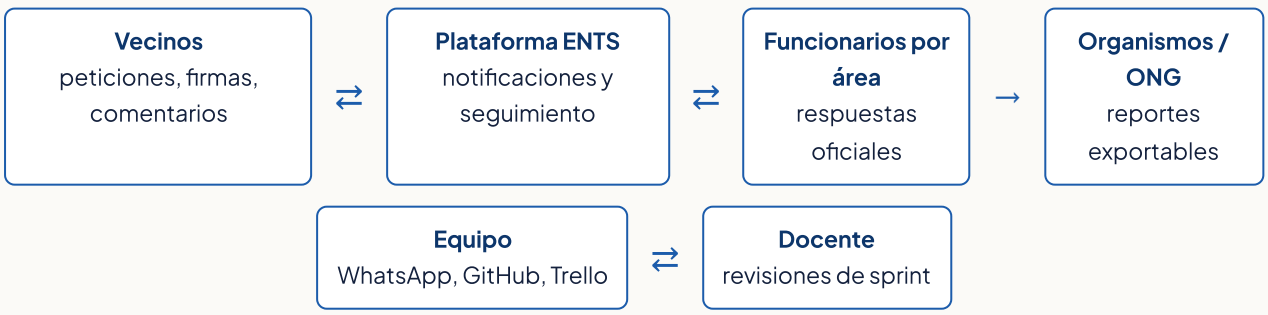
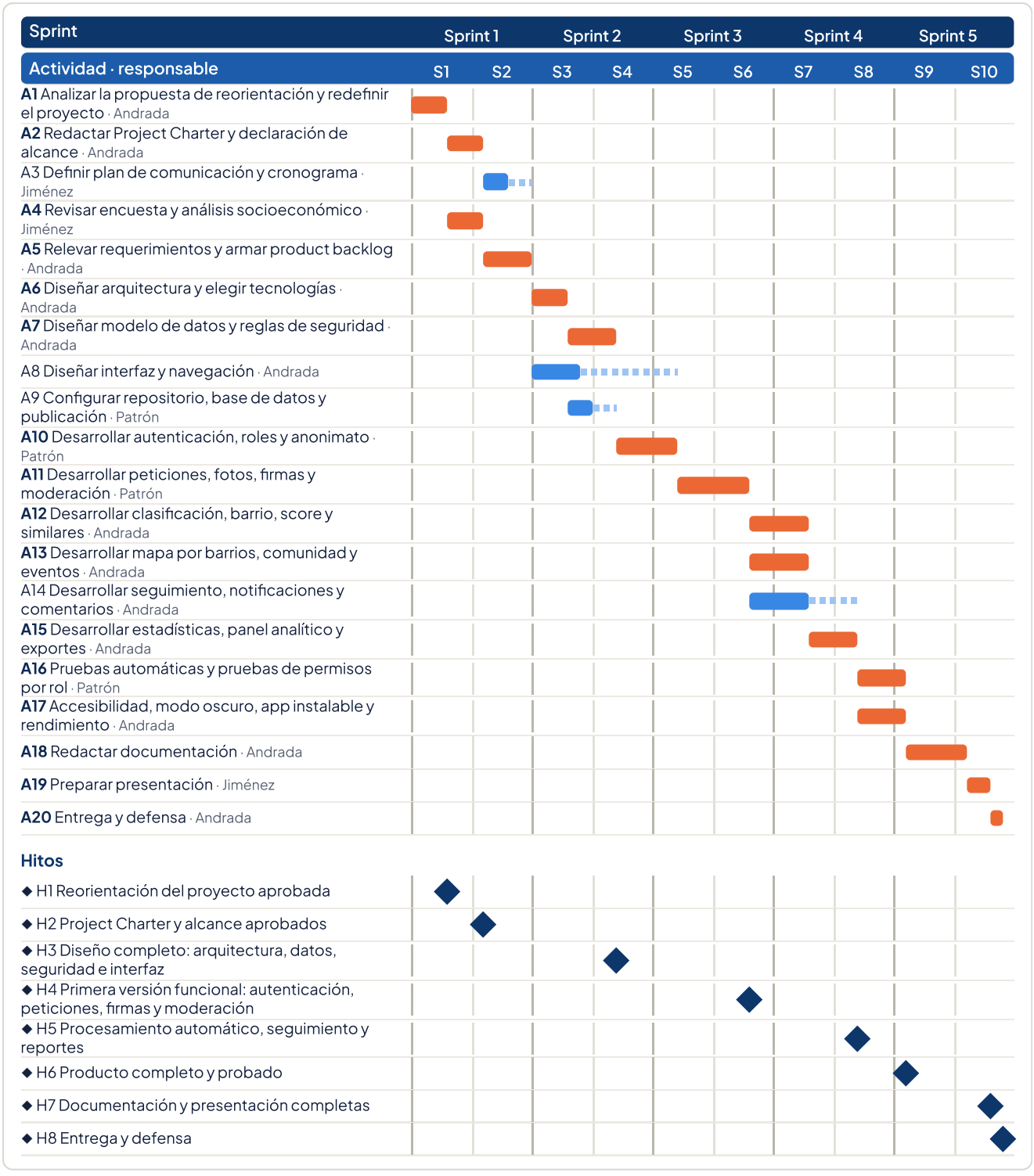


Figura 2. Flujo de la comunicación.

3.8 Diagrama de Gantt

Cronograma de 49 días hábiles (≈ 10 semanas) calculado a partir de la lista de actividades; en naranja, la ruta crítica. Las barras muestran el responsable de cada actividad y las líneas más marcadas, el comienzo de cada sprint de 2 semanas. Las fechas reales de cada entrega de código figuran en el Anexo A.



Ruta crítica: A1 → A2 → A4 → A5 → A6 → A7 → A10 → A11 → A12 → A13 → A15 → A16 → A17 → A18 → A19 → A20.

3.9 Requerimientos y product backlog

Requerimientos funcionales (RF)

Describen lo que el sistema debe hacer. Prioridad **Alta** = imprescindible para que el sistema cumpla su propósito; **Media** = aporta valor importante; **Baja** = mejora deseable que se construye al final.

ID	Requerimiento	Prioridad
RF01	Registrar usuarios con correo y contraseña o con Google, y recuperar la contraseña por correo.	Alta
RF02	Elegir el tipo de cuenta al registrarse: vecino o funcionario (esta última queda sujeta a aprobación).	Alta
RF03	Crear peticiones con título, descripción, hasta 3 fotos y ubicación en el mapa.	Alta
RF04	Sugerir automáticamente la categoría según el texto y el barrio según la ubicación.	Alta
RF05	Calcular automáticamente el score de urgencia (0 a 100) de cada petición.	Alta
RF06	Advertir peticiones similares antes de crear una nueva.	Alta
RF07	Moderar las peticiones antes de publicarlas: aprobar, rechazar, editar o eliminar.	Alta
RF08	Firmar una petición publicada y retirar la firma; ordenar el inicio por prioridad según las firmas.	Alta
RF09	Elevar automáticamente al área correspondiente las peticiones que alcanzan 50 firmas.	Media
RF10	Registrar respuestas oficiales y la etapa de seguimiento, con foto opcional.	Alta
RF11	Notificar al autor y a los firmantes cada novedad, y permitir seguir barrios.	Media
RF12	Comentar en forma anónima, denunciar comentarios y moderarlos.	Media
RF13	Mostrar el mapa por barrios coloreado por cantidad de peticiones, firmas o urgencia.	Media
RF14	Ofrecer estadísticas públicas anónimas y un panel analítico para administradores.	Media
RF15	Exportar reportes de peticiones y barrios a Excel y PDF.	Media
RF16	Asignar un área a cada funcionario y mostrarle una bandeja con las peticiones de esa área.	Media
RF17	Permitir solo al dueño del proyecto cambiar el rol de las cuentas.	Baja
RF18	Publicar en el muro vecinal y consultar la agenda de eventos por barrio.	Baja
RF19	Permitir solo al dueño definir qué paneles ve y modifica cada rol (permisos por panel).	Baja
RF20	Permitir a los moderadores revisar peticiones pendientes, comentarios denunciados y publicaciones.	Media
RF21	Elegir el barrio donde vive cada vecino (a mano o con su ubicación), cambiarlo como máximo una vez cada 30 días y mostrar primero lo de ese barrio en el inicio, el mapa, la comunidad, los eventos y los avisos.	Media

ID	Requerimiento	Prioridad
RF22	Resumir la actividad del barrio del vecino comparada con el promedio de la ciudad y mostrar un ranking de los barrios más activos y de los que más resolvieron.	Baja

Requerimientos no funcionales (RNF)

Describen cualidades que el sistema debe tener, independientemente de lo que hace.

ID	Atributo	Requerimiento
RNF01	Privacidad	Anonimato entre vecinos: la identidad del autor y de los firmantes solo la ven los funcionarios del área y los administradores. El barrio donde vive cada vecino no se muestra a nadie, salvo que la persona elija mostrar "Vecino/a de [barrio]" en sus comentarios.
RNF02	Seguridad	Las reglas de acceso se aplican en la base de datos (RLS), no solo en la pantalla; toda la comunicación usa HTTPS.
RNF03	Usabilidad	Interfaz en español, adaptable a celular, con modo claro y oscuro.
RNF04	Accesibilidad	Navegación con teclado, foco visible, respeto del movimiento reducido y tablas alternativas a los gráficos (WCAG 2.2).
RNF05	Rendimiento	Cada página se descarga recién al visitarla; en celulares se desactivan las animaciones continuas.
RNF06	Disponibilidad	Aplicación instalable (PWA) que abre aun con conexión inestable.
RNF07	Mantenibilidad	Revisión automática del código (lint), pruebas automáticas y esquema SQL versionado.
RNF08	Legal	Política de privacidad publicada y cumplimiento de la Ley 25.326 de Protección de Datos Personales.

Product backlog (historias de usuario)

Cada historia de usuario (HU) expresa una necesidad con la forma *"Como [rol], quiero [acción] para [beneficio]"*, con criterios de aceptación verificables. La última columna indica qué requerimientos funcionales la implementan (trazabilidad).

ID	Historia de usuario	Criterios de aceptación	Prioridad	RF
HU01	Como **vecino**, quiero registrarme e ingresar, también con Google para participar en la plataforma.	Formulario con nombre, apellido, correo y contraseña; errores en español; recuperación de contraseña por correo.	Alta	RF01, RF02
HU02	Como **vecino**, quiero crear una petición con fotos y ubicación para visibilizar un problema de mi barrio.	Categoría y barrio sugeridos automáticamente; hasta 3 fotos; queda pendiente hasta que se modere.	Alta	RF03, RF04, RF05
HU03	Como **vecino**, quiero ver si ya existe un pedido parecido para firmarlo en lugar de repetirlo.	Mientras escribo se muestran hasta 3 peticiones similares, con barrio, firmas y distancia.	Alta	RF06

ID	Historia de usuario	Criterios de aceptación	Prioridad	RF
HU04	Como **vecino**, quiero firmar una petición para darle más prioridad.	Una firma por persona; puedo retirarla; el orden y el color de prioridad se actualizan.	Alta	RF08, RF09
HU05	Como **moderador**, quiero revisar las peticiones antes de publicarlas para evitar contenido indebido o con datos personales.	Aprobar, rechazar, editar o eliminar; las alertas automáticas se muestran primero.	Alta	RF07, RF20
HU06	Como **funcionario**, quiero ver las peticiones de mi área ordenadas por prioridad y responderlas para atender primero lo más urgente.	Bandeja filtrada por área; respuesta con etapa, mensaje y foto.	Alta	RF10, RF16
HU07	Como **vecino**, quiero seguir el avance de una petición hasta que se resuelva para saber que mi reclamo fue atendido.	Línea de tiempo con 5 etapas y respuestas oficiales; aviso en cada cambio.	Alta	RF10, RF11
HU08	Como **vecino**, quiero ver en un mapa los problemas de cada barrio para conocer qué pasa en mi zona.	Mapa con 111 barrios coloreados; filtro por barrio y opción de seguirlo.	Media	RF11, RF13
HU09	Como **organismo**, quiero obtener reportes de qué se pide, dónde y con qué urgencia para decidir qué atender y rendir cuentas.	Estadísticas por categoría, barrio y semana; exportes a Excel y PDF.	Media	RF14, RF15
HU10	Como **vecino**, quiero comentar en forma anónima para aportar datos sobre un problema.	Se bloquean teléfonos, correos y documentos; un comentario con 3 denuncias se oculta.	Media	RF12
HU11	Como **vecino**, quiero participar del muro vecinal y de la agenda de eventos para organizarme con mis vecinos.	Publicaciones y eventos filtrados por barrio.	Baja	RF18
HU12	Como **dueño**, quiero asignar el rol de cada cuenta para controlar quién modera, responde o administra.	Solo el dueño ve y usa el selector de rol; la base lo vuelve a verificar.	Baja	RF17
HU13	Como **dueño**, quiero definir qué paneles ve y modifica cada rol para adaptar los accesos sin cambiar el código.	Pantalla "Permisos" con cada rol y sus paneles; interruptores de ver y modificar que se guardan al instante.	Baja	RF19
HU14	Como **vecino**, quiero elegir el barrio donde vivo para ver primero lo que pasa cerca de mí.	Se pregunta en el primer ingreso (se puede dejar para después); el inicio agrupa las peticiones en mi barrio, cerca y resto de la ciudad; el mapa, la comunidad, los eventos y los avisos arrancan por mi barrio; se cambia una vez cada 30 días.	Media	RF21, RF22

Todos los requerimientos funcionales están cubiertos por al menos una historia de usuario (detalle en el Entregable 2, sección 6).

3.10 Arquitectura del sistema

Se mantiene la separación en capas del diseño original (modelo-vista-controlador), pero se simplifica la infraestructura: en lugar de un backend propio en Node.js/C# con MySQL, Docker y Heroku, se utiliza un **backend como servicio** (Supabase, sobre PostgreSQL) que provee base de datos, autenticación, almacenamiento de archivos y una API generada a partir del esquema. Esto reduce la infraestructura a mantener y concentra la lógica de negocio crítica en la base de datos, donde no puede ser alterada desde el navegador.

Capa	Tecnología	Responsabilidad
Vista	React 19, React Router, Bootstrap 5 con tema propio, Leaflet	Páginas y componentes; interacción; validaciones inmediatas (categoría, moderación, similares).
Controlador	Servicios de la app (`src/services`) y funciones RPC de la base	Orquestación de casos de uso: crear, firmar, responder, moderar, reportar.
Modelo	PostgreSQL en Supabase, con RLS y disparadores	Datos, reglas de acceso, cálculo del score, notificaciones, prioridad y elevación.

Paquetes del código

Paquete	Contenido
`src/pages`	Una pantalla por archivo: Inicio, Mapa, Comunidad, Eventos, Petición, Nueva petición, Bandeja, Estadísticas, Moderación, Usuarios, Perfil, Documentos, Permisos y Acceso.
`src/components`	Piezas reutilizables: tarjeta de petición, seguimiento, comentarios, gráficos, selectores de fotos y ubicación, menús.
`src/services`	Acceso a datos: peticiones, participación (seguimiento, comentarios, roles) y autenticación.
`src/utils`	Lógica pura y probada: prioridad, clasificador, barrios, moderación, exportación, zonas del mapa.
`supabase/schema.sql`	Esquema completo: tablas, políticas, funciones y disparadores.

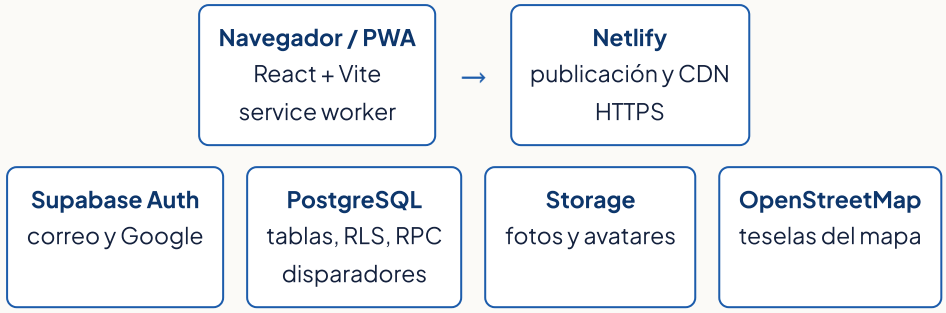


Figura 3. Diagrama de despliegue: el cliente se descarga desde Netlify y se comunica por HTTPS con los servicios de Supabase y con OpenStreetMap. GitHub dispara la publicación automática en cada versión.

3.11 Modelo de datos

Base relacional en PostgreSQL con 14 tablas. Todas tienen seguridad por fila activada.

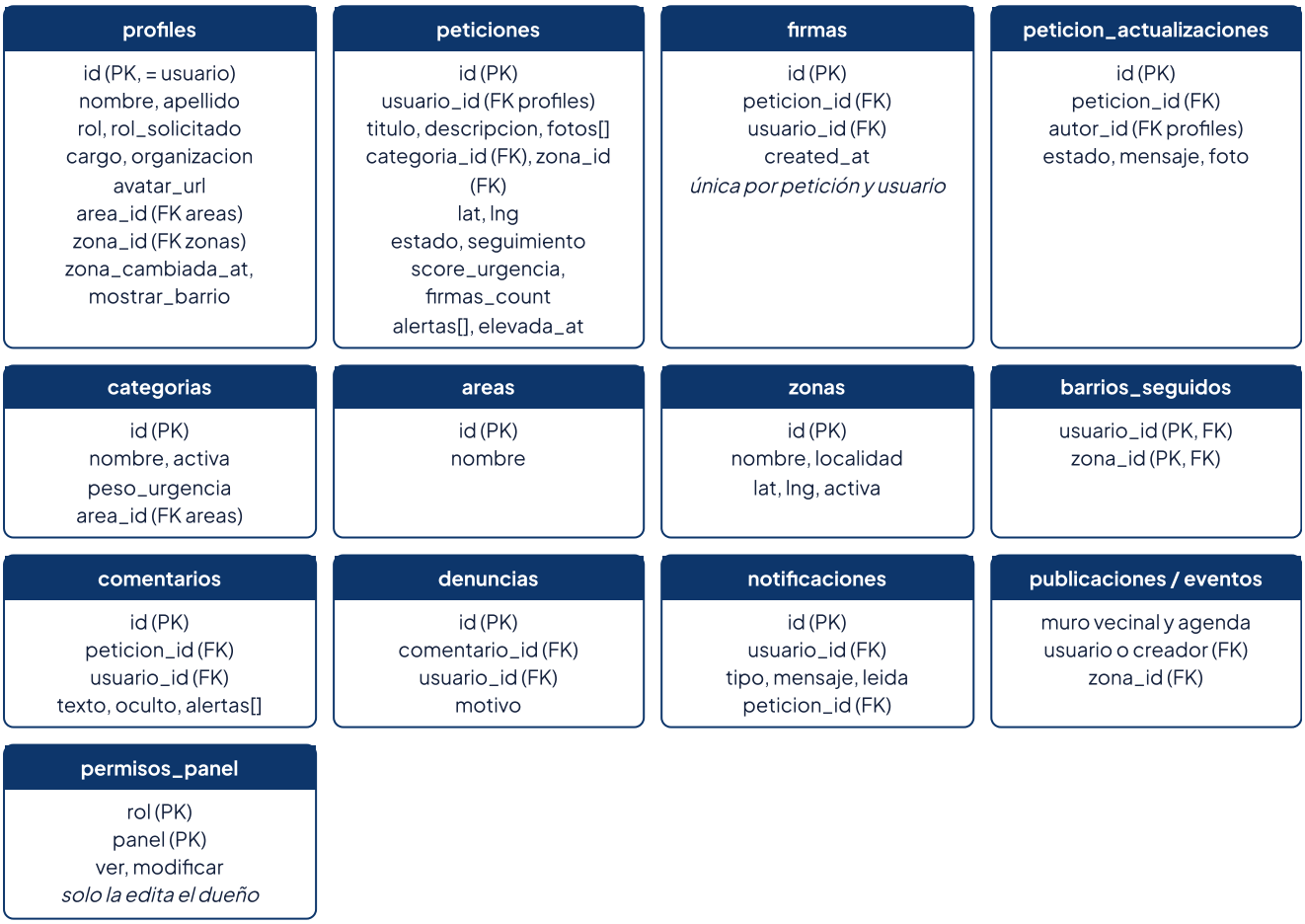


Figura 4. Diagrama entidad-relación (simplificado). Relaciones principales: una petición pertenece a un usuario, una categoría y un barrio; tiene muchas firmas, actualizaciones y comentarios; cada categoría pertenece a un área; cada perfil puede indicar el barrio donde vive.

Diagrama de estados de una petición

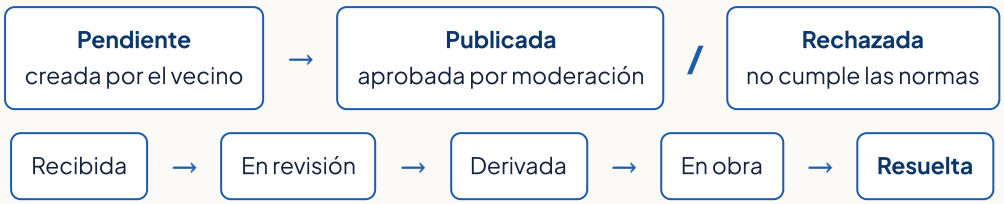


Figura 5. Estados de moderación (arriba) y etapas de seguimiento de una petición publicada (abajo). Al alcanzar 50 firmas se registra además la elevación al área.

3.12 Procesamiento automático de datos

Clasificación por categoría

Cada una de las 16 categorías tiene raíces de palabras clave (por ejemplo, Salud: hospital, dispensario, pediatr…). El texto se normaliza (minúsculas, sin acentos) y se cuentan coincidencias al inicio de palabra; las del título valen el doble. Gana la categoría con más puntaje; si hay empate total, no se sugiere y elige la persona.

Detección del barrio

Con la ubicación marcada (o la del dispositivo) se elige el barrio cuyo centro está más cerca, hasta 3 km, entre 111 barrios de Córdoba y Villa Allende cuyas coordenadas se obtuvieron de OpenStreetMap.

Barrio del vecino y barrios cercanos

Cada cuenta puede elegir el barrio donde vive, de una lista o con la ubicación del dispositivo (se usa la misma detección por cercanía). Con ese dato, el inicio agrupa las peticiones en tres bloques: **en tu barrio**, **cerca tuyo** (barrios cuyo centro está a menos de 3 km) y **en el resto de la ciudad**. La función `resumen_mi_barrio()` compara el barrio con el promedio de la ciudad (peticiones activas, firmas, resueltas del mes, urgencia y la categoría que más se repite) y `ranking_barrios()` calcula los barrios más activos y los que más resolvieron en el último mes.

Score de urgencia (0 a 100)

Componente	Puntos
Gravedad de la categoría (p. ej. Salud 35, Cultura 12)	hasta 35
Palabras de riesgo en el texto (peligro, accidente, inundación, sin agua, chicos…)	8 c/u, hasta 30
Apoyo: 20 × ln(1 + firmas) / ln(51)	hasta 20
Velocidad: 2 por firma de los últimos 7 días	hasta 10
Evidencia fotográfica	5

Se calcula en la base de datos al crear, editar y con cada firma. Niveles: urgente (70 o más), media (40 a 69) y baja (menos de 40). Ejemplo: "Cables caídos, peligro para los chicos" (Servicios públicos) obtiene 53; con 10 firmas y una foto llega a 70.

Prioridad por firmas

Nueva (0–4), baja (5–9), media (10–19), alta (20–49) y máxima (50 o más). El inicio ordena por firmas, luego por urgencia y por fecha. A las 50 firmas la petición se eleva y se notifica a los funcionarios del área y a los administradores.

Detección de peticiones similares

Se extraen las palabras clave (sin conectores ni términos genéricos como "calle" o "barrio") y se comparan por similitud de trigramas (extensión `pg_trgm`). Se sugieren hasta 3 peticiones con parecido ≥ 0,32, o ≥ 0,22 si están a menos de 800 metros.

Moderación automática

Se detectan correos, teléfonos, números de documento y lenguaje ofensivo. En comentarios, los datos personales se bloquean y el lenguaje ofensivo queda oculto hasta revisión; en peticiones, se muestran alertas al moderador.

3.13 Autenticación, roles y seguridad

Rol	Qué puede hacer (configuración inicial)
Vecino	Crear, firmar y comentar peticiones; elegir su barrio y seguir otros; publicar en el muro; ver mapa, eventos y estadísticas públicas. Es anónimo para los demás vecinos.
Funcionario	Todo lo del vecino, más la bandeja de su área: ve autor y firmantes, responde y cambia la etapa de las peticiones de su área. Lo aprueba un administrador.
Moderador	Todo lo del vecino, más revisar peticiones pendientes, comentarios denunciados y publicaciones del muro. No administra usuarios ni ve el panel analítico.
Administrador	Todo lo del moderador y del funcionario (en todas las áreas), más el directorio de usuarios, la asignación de áreas, la agenda de eventos y el panel analítico.
Dueño (CEO)	Una única cuenta de administrador que además cambia el rol de cada cuenta, define los permisos por panel y accede a los documentos privados del proyecto.

Permisos por panel. La tabla anterior describe la configuración inicial. Desde la pantalla "Permisos", el dueño decide para cada rol y cada panel (mis solicitudes, mapa, comunidad, estadísticas, eventos, perfil, bandeja, moderación, usuarios, panel analítico y documentos) si puede verlo y si puede modificar en él. La configuración se guarda en la tabla `permisos_panel` y la base de datos la aplica en sus reglas y funciones, por lo que no alcanza con ocultar un botón en la pantalla.

- **Autenticación:** correo y contraseña o Google (OAuth); recuperación de contraseña por correo; sesiones gestionadas por Supabase Auth.
- **Seguridad por fila (RLS):** cada tabla define quién puede leer, crear, modificar o borrar cada fila.
- **Anonimato por columna:** la columna del autor de una petición no es legible para los vecinos; las identidades solo se obtienen mediante funciones que verifican el rol.
- **Funciones seguras:** las operaciones sensibles (responder, moderar, cambiar rol o permisos, estadísticas de administración) verifican el rol y los permisos por panel dentro de la base.

- **Barrio del vecino:** no se muestra a otros vecinos, salvo que la persona active "Vecino/a de [barrio]" en sus comentarios. Solo se puede cambiar una vez cada 30 días, y ese límite lo controla la base (`elegir_mi_barrio`) para que nadie se "mude" de barrio para inflar firmas o rankings.
- **Validaciones del servidor:** el score, la etapa inicial y las alertas los fija la base, aunque el cliente intente enviar otros valores.
- **Transporte:** toda la comunicación usa HTTPS.

Las reglas se verificaron simulando usuarios de cada rol dentro de transacciones que luego se deshacen: un vecino no puede responder como funcionario, ni publicar un teléfono, ni cambiar la etapa, ni ver la moderación, ni cambiar roles.

3.14 Reportes y evaluación de impacto

La evaluación de impacto se apoya en los datos que la plataforma registra en cada paso del ciclo:

Indicador	Cómo se mide
Problemáticas activas	Peticiones publicadas por categoría, barrio y semana.
Apoyo comunitario	Firmas totales, por día y velocidad de crecimiento.
Urgencia	Distribución alta / media / baja y urgencia promedio por barrio.
Respuesta institucional	Peticiones por etapa de seguimiento y tiempo promedio de revisión.
Resolución	Cantidad y proporción de peticiones resueltas.
Participación	Cuentas activas en 30 días, publicaciones y comentarios.
Actividad por barrio	Ranking mensual de los barrios más activos y de los que más resolvieron; resumen de cada barrio frente al promedio de la ciudad.

Las estadísticas públicas muestran solo agregados anónimos. El panel analítico de administradores agrega datos de cuentas y moderación, y permite exportar las peticiones (con área, barrio, etapa, prioridad, firmas y urgencia) y el resumen por barrio a Excel, y la vista completa a PDF, para presentar ante organismos y ONG.

3.15 Pruebas y calidad

- **Pruebas automáticas:** 49 pruebas con Vitest sobre prioridad, orden del inicio, clasificación, detección de barrio, moderación de textos, áreas del mapa, exportación y permisos de cada rol. Las pruebas detectaron y permitieron corregir un error real en la detección de lenguaje ofensivo.
- **Pruebas de permisos:** escenarios por rol ejecutados en la base de datos.
- **Revisión de código:** oxlint sin errores y compilación de producción antes de cada publicación.
- **Accesibilidad:** enlace para saltar al contenido, foco visible, etiquetas asociadas a los campos, contraste en modo claro y oscuro, versión en tabla de cada gráfico y respeto de la preferencia de movimiento reducido.
- **Datos de demostración:** para exponer la plataforma con actividad se cargan 300 cuentas, 200 peticiones, firmas, comentarios, publicaciones y eventos ficticios, con fotos ilustrativas

(`supabase/demo`). Las cuentas usan el dominio `demo.ents.invalid`, no tienen contraseña y se eliminan con un solo script sin tocar los datos reales.

- **Limpieza del código:** se quitaron la dependencia `react-bootstrap` (sin uso), imágenes de la plantilla, estilos y funciones que nadie usaba y comentarios redundantes.
- **Rendimiento:** cada página se descarga recién al visitarla; en celulares se desactivan animaciones continuas y desenfoques; las listas largas no se dibujan fuera de pantalla.

3.16 Ambiente de implementación y despliegue

Herramienta	Uso
React 19 + Vite 8	Desarrollo y compilación del cliente web.
Bootstrap 5 + tema propio	Estilos, modo claro y oscuro.
Leaflet + OpenStreetMap	Mapa vecinal y selector de ubicación.
Supabase (PostgreSQL)	Base de datos, autenticación, almacenamiento y API.
Netlify	Publicación continua desde GitHub con HTTPS.
Git + GitHub	Control de versiones; ramas y Pull Requests.
Vitest + oxlint	Pruebas automáticas y revisión de código.
Node.js 22	Entorno de desarrollo local.

Ambientes: desarrollo local (`npm run dev`) conectado a la base en la nube; producción en Netlify, que publica automáticamente cada versión de la rama principal del repositorio. El esquema completo de la base se versiona en `supabase/schema.sql` para poder recrearla.

4 Bibliografía

Obra	Autor	Edición	Editorial
Guía de los fundamentos para la dirección de proyectos (Guía del PMBOK®), 7.ª edición	Project Management Institute	2021	Project Management Institute
Ingeniería de Software, 9.ª edición	Ian Sommerville	2011	Pearson Educación
Ingeniería del software: un enfoque práctico, 7.ª edición	Roger S. Pressman	2010	McGraw-Hill
La Guía de Scrum	Ken Schwaber y Jeff Sutherland	2020	Scrum.org / Scrum Inc.
Clean Code: A Handbook of Agile Software Craftsmanship	Robert C. Martin	2008	Prentice Hall
Web Content Accessibility Guidelines (WCAG) 2.2	W3C	2023	World Wide Web Consortium

5 Fuentes de información

- Docentes de la cátedra Práctica Profesionalizante del Instituto Cervantes.
- Documentación previa del equipo: presentación "Everyone Needs To Smile", entregables de gestión y documento de Práctica Profesionalizante (versión de donaciones).
- Encuesta de necesidades a vecinos y análisis socioeconómico de Córdoba realizados por el equipo.
- OpenStreetMap y su servicio de búsqueda Nominatim: ubicación de los 111 barrios de Córdoba y Villa Allende.
- Documentación oficial de React (react.dev), Vite (vite.dev), Supabase (supabase.com/docs), PostgreSQL (postgresql.org/docs), Leaflet (leafletjs.com), Netlify (docs.netlify.com) y MDN Web Docs (aplicaciones web progresivas).

6 Anexo

A. Hitos reales del repositorio

Fecha	Entrega
04/10/2026	Configuración inicial del proyecto; autenticación, peticiones, firmas y moderación.
11/10/2026	Panel vecinal conectado a Supabase; roles y anonimato; ingreso con Google y recuperación de contraseña; política de privacidad.
12/10/2026	Estadísticas y mapa por barrios; 111 barrios; prioridad por firmas, seguimiento, comentarios y notificaciones; bandeja por área, exportes, PWA, modo oscuro y pruebas; cambio de roles; rendimiento en celulares; score de urgencia automático; rol moderador y permisos por panel definidos por el dueño.
13/10/2026	Corrección: borrar un comentario ahora persiste (función de la base que verifica autor o permiso de moderación).
18/10/2026	Datos de demostración; inicio y panel de moderación más compactos; barrio del vecino (elección, inicio por barrio, mapa centrado, ranking e insignia); limpieza del código.

B. Fórmula del score de urgencia (extracto del esquema)

```
gravedad  := peso_urgencia_de_la_categoria * 35 / 30;          -- hasta 35
riesgo    := least(8 * palabras_de_riesgo_encontradas, 30);    -- hasta 30
apoyo     := least(20, 20 * ln(1 + firmas) / ln(51));          -- hasta 20
velocidad := least(10, 2 * firmas_de_los_ultimos_7_dias);      -- hasta 10
evidencia := case when tiene_fotos then 5 else 0 end;          -- 5
score     := least(100, round(gravedad + riesgo + apoyo + velocidad + evidencia));
```

C. Puesta en marcha local

```
git clone https://github.com/andradaj09/ENTS.git
cd ENTS
npm install
# crear .env con VITE_SUPABASE_URL y VITE_SUPABASE_ANON_KEY
npm run dev        # http://localhost:5173
npm test           # pruebas automáticas
npm run build      # versión de producción
```

D. Enlaces

- Aplicación publicada: https://entscordoba.netlify.app
- Repositorio (privado): https://github.com/andradaj09/ENTS